

INTRODUCCIÓN PYTHON

Preparado por Mgtr. Tomás Calderón



Python: Un lenguaje de alto nivel, intuitivo y legible

¿Qué es Python?

- Lenguaje interpretado: No requiere compilación antes de ejecutarse, lo que permite interactividad y pruebas rápidas en tiempo real.
- Diseñado para la legibilidad: Su filosofía enfatiza la claridad del código, reduciendo la cantidad de líneas en comparación con C, C++ o Java.
- Multipropósito: Utilizado en IA, ciencia de datos, desarrollo web, automatización y sistemas embebidos.

Un poco de historia

Creado por Guido van Rossum en diciembre de 1989 como un proyecto navideño personal. El nombre rinde homenaje al grupo cómico británico Monty Python.

El Zen de Python (import this)

- *La legibilidad cuenta.*
- *Lo explícito es mejor que lo implícito.*
- *Lo simple es mejor que lo complejo.*
- *Si la implementación es difícil de explicar, es una mala idea.*
- *Si es fácil de explicar, puede que sea una buena idea.*

La indentación obligatoria define bloques de código

Reglas Generales de Sintaxis

1. Sensible a Mayúsculas y Minúsculas (Case Sensitive):
Las variables 'edad' y 'Edad' son distintas para Python.
2. Sin Llaves {} ni Puntos y Comas ;:
A diferencia de C/C++ o Java, Python estructura su código puramente a través de espacios o tabulaciones.
3. Indentación Obligatoria:
Un bloque de código (por ejemplo, dentro de un condicional o función) se define por su nivel de sangrado o indentación.
4. Comentarios en Código:
 - # se utiliza para comentarios de una sola línea.
 - """ Comentario """ para comentarios multilínea o de documentación.

Sintaxis Limpia vs Tradicional

```
# Comentario en Python
edad = 20

if edad >= 18:
    # Bloque indentado
    print("Es mayor de edad")
else:
    print("Es menor de edad")
```

Tipos de datos dinámicos y colecciones

Python infiere el tipo de dato automáticamente al asignarse un valor. Las variables son dinámicas.

Categoría / Tipo	Ejemplo en Python	Características y Uso
Entero (int)	<code>temperatura = 27</code>	Números enteros sin punto decimal. Útiles para conteos o IDs.
Decimal (float)	<code>voltaje = 3.3</code>	Números de punto flotante. Crucial para valores precisos de sensores.
Cadena (str)	<code>dispositivo = "NodeMCU"</code>	Secuencias de caracteres para texto. Se delimitan con comillas.
Booleano (bool)	<code>estado_led = True</code>	Valores lógicos: True o False. Ideales para banderas de control.
Lista (list)	<code>lecturas = [25, 27, 26]</code>	Colección ordenada y mutable de elementos (se puede modificar).
Diccionario (dict)	<code>sensor = {'id': 1, 'temp': 25}</code>	Colección mutable de pares clave-valor (clave única para acceso rápido).

Operaciones de Entrada, Salida y Casting

Funciones Básicas de E/S

- `print()`: Envía información a la consola. Puede imprimir cadenas de texto, variables o expresiones complejas separadas por comas.
- `input()`: Solicita y lee información ingresada por el usuario desde la terminal.

⚠️ Importante: La función `input()` siempre retorna un dato de tipo cadena (string o str). No se puede realizar aritmética directamente sobre un valor devuelto por `input()`.

Conversión de Datos (Casting)

Para procesar números ingresados por el usuario, debemos convertirlos explícitamente usando `int()` para enteros, `float()` para decimales, o `str()` para cadenas.

Ejemplo de E/S y Conversión

```
# Imprimir un saludo
nombre = "Juan"
print("Hola,", nombre)

# Entrada de usuario y casting
edad_texto = input("Ingrese su edad: ")
edad = int(edad_texto) # Convierte str a int

# Aritmética directa con el entero
siguiente_anio = edad + 1
print("El próximo año tendrás:", siguiente_anio)
```

Condicionales, bucles iterativos y control de excepciones

Control de Flujo Estándar

- Estructura Condicional (if, elif, else):
Permite evaluar múltiples caminos lógicos según condiciones booleanas.
- Bucle for:
Itera sobre una secuencia (como una lista o un rango generado con range()). Excelente para iteraciones definidas.
- Bucle while:
Repite código continuamente mientras una condición lógica sea verdadera. Ideal para bucles de monitoreo continuo de sensores.

Gestión de Errores (Try / Except)

En desarrollo de software y adquisición de datos, los errores pueden ocurrir en tiempo de ejecución (por ejemplo, entradas de usuario corruptas).

Estructura try-except:
Evita que el programa se cuelgue. 'Intenta' ejecutar un fragmento de código peligroso, y si falla, ejecuta un bloque alternativo sin detener la aplicación.

```
try:  
    resultado = 10 / valor_sensor  
except ZeroDivisionError:  
    print("Error: Sensor inactivo o en cero")
```

Thonny IDE: El entorno ideal para iniciar

Thonny es un entorno de desarrollo integrado (IDE) desarrollado específicamente para enseñar y aprender programación en Python de forma visual.

Instalación Simple

Viene con Python 3.14 incorporado en su instalador oficial.

Evita problemas comunes de configuración, como añadir rutas al sistema (PATH) o lidiar con múltiples intérpretes de Python que puedan causar conflictos en el salón de clases.

Interfaz sin Distracciones

Su diseño es minimalista y está limpio de funciones abrumadoras para principiantes.

Permite a los estudiantes escribir su código y ejecutarlo inmediatamente (usando la tecla F5), eliminando la confusión visual de entornos empresariales complejos.

Depuración Amigable

Permite la ejecución paso a paso (Ctrl+F5) sin necesidad de configurar puntos de interrupción complejos.

Muestra de manera explícita cómo Python evalúa las expresiones y cómo cambian las variables durante la ejecución.

Guía práctica: Cómo iniciar un programa en Thonny

1. Escribir y Guardar

- Abre Thonny y escribe tu primer programa en el área del editor (por ejemplo, `print("Hola UTP")`).
- Ve a 'Archivo' -> 'Guardar' o presiona Ctrl+S.
- Guarda tu script con la extensión oficial '.py'.

2. Ejecución Directa

- Haz clic en el botón de reproducción ('Ejecutar') o simplemente presiona la tecla F5.
- Observa la consola ('Shell') en la parte inferior de Thonny para ver el resultado de la salida del script en tiempo real.

3. Activar Herramientas

- Selecciona 'Ver' -> 'Variables' en la barra superior para ver tu tabla de memoria.
- Ejecuta con Ctrl+F5 para depurar paso a paso (F7 para pasos cortos).
- Thonny resaltará en azul cómo evalúa paso a paso cada operación.